

Esqueleto de proyecto:

Normalizador estadístico vectorizado (NASM + C)

Este es el punto de partida para el proyecto “Programación Vectorial en Ensamblador x86-64 (NASM/Linux)”. **Aquí no está la solución**: contiene la estructura, las firmas de las funciones y un ejemplo completo por versión (`sum_array`) que sirve de patrón. El resto de las funciones (`compute_stats` , `normalize_array`) están marcadas con `TODO` y deben ser implementadas por el estudiante, tanto en la versión escalar como en la vectorial.

Estructura

```
.
├─ Makefile
├─ include/
│   └─ stats.h           # Firmas compartidas por ambas versiones
├─ src/
│   └─ driver.c          # Programa principal (E/S, timing, impresión)
├─ asm/
│   └─ scalar/
│       └─ stats_scalar.asm # Versión escalar (SSE escalar)
│       └─ vector/
│           └─ stats_vector.asm # Versión vectorial (AVX2)
├─ tools/
│   └─ gen_input.py       # Genera archivos de entrada de prueba
│   └─ verify_reference.py # Verifica resultados contra referencia en Python puro
└─ data/                  # Se crea al compilar: entradas/salidas .dat
```

Requisitos

- Linux con CPU compatible con AVX2 (verificar con `lscpu | grep avx2`).
- `nasm` , `gcc` , `make` , `python3` .
- `gdb` y, opcionalmente, `perf` (paquete `linux-tools`) para las partes de verificación y medición de rendimiento del proyecto.

Compilar

```
bash
make
```

Genera `bin/norm_scalar` y `bin/norm_vector` : dos ejecutables que comparten el mismo `driver.c` pero enlazan con kernels distintos (`obj/stats_scalar.o` u `obj/stats_vector.o`).

Generar datos de prueba

```
bash
python3 tools/gen_input.py 1000000 data/input.dat random
python3 tools/gen_input.py 8 data/input_small.dat random
python3 tools/gen_input.py 1000 data/input_constant.dat constant
python3 tools/gen_input.py 0 data/input_empty.dat random
```

Genere tambien casos con `N` no multiplo de 8 (por ejemplo 7, 15, 1001) para probar el manejo del remanente.

Ejecutar

```
bash
./bin/norm_scalar data/input.dat data/output_scalar.dat 30
./bin/norm_vector data/input.dat data/output_vector.dat 30
```

El tercer argumento es el numero de repeticiones del kernel, usado para promediar el tiempo medido con `clock_gettime` (util para sus mediciones de rendimiento con distintos tamanos de `N`).

Cada corrida tambien escribe `data/output_scalar.dat.stats.txt` (o `_vector.dat.stats.txt`) con un resumen en texto plano de los estadisticos y el tiempo del kernel.

Verificar correctud

```
bash
python3 tools/verify_reference.py data/input.dat data/output_scalar.dat.stats.txt
python3 tools/verify_reference.py data/input.dat data/output_vector.dat.stats.txt
```

Lo que debe implementar el estudiante

- . `asm/scalar/stats_scalar.asm`: completar `compute_stats` y `normalize_array` con instrucciones escalares (`movss` , `addss` , `subss` , `mulss` , `divss` , `sqrts` , `comiss` , etc.).
- . `asm/vector/stats_vector.asm`: completar `compute_stats` y `normalize_array` con AVX2 (`vmovaps` / `vmovups` , `vaddps` , `vsubps` , `vmulps` , `vdivps` , `vminps` , `vmaxps` , `vbroadcastss` , reduccion horizontal), **manejando el remanente** igual que en `sum_array` de ejemplo.
- . Generar sus propios archivos de prueba con `gen_input.py` para los casos borde exigidos en la propuesta (N=0, N=1, N no multiplo de 8, valores constantes, valores negativos/extremos).
- . Usar GDB para inspeccionar registros YMM y memoria en un caso pequeno, como se pide en la propuesta (ver ejemplo mas abajo).
- . Medir tiempos con distintos tamanos de `N` (use el argumento de repeticiones del driver) y, opcionalmente, `perf stat` .

Notas de depuracion con GDB

Los binarios se compilan con simbolos de depuracion (`-g` en gcc y `-g -F dwarf` en nasm), por lo que se puede poner breakpoints directamente en las etiquetas del ensamblador:

```
bash
gdb --args ./bin/norm_vector data/input_small.dat data/out.dat 1
(gdb) break normalize_array
(gdb) run
(gdb) info registers ymm0
(gdb) stepi
(gdb) x/8fw &out[0]
```

(La sintaxis exacta para imprimir un YMM completo como 8 floats depende de la version de GDB instalada: pruebe `info registers ymm0`, `print ymm0.v8fload </code>`, `o <code> p/ xymm0` segun lo que este disponible en su laboratorio.)